3iL

Programmation professionnelle

3IL INGÉNIEURS · RODEZ · 3IL I2 · SECTION PAR APPRENTISSAGE (PROMO 24) · 2026-2027

PROJET

Le projet, décrire une image à qui ne la voit pas

Le sujet du projet, réalisé en binôme dans le cadre d'un défi d'accessibilité et d'audiodescription. Il s'agit d'une application web qui décrit à voix haute une image déposée par l'utilisateur, pour les personnes qui ne peuvent pas la voir. Elle utilise Spring Boot et LangChain4J, et l'agent est placé derrière une interface.

L'usage de l'assistant est autorisé et même attendu ; relisez la partie « Ce que l'assistant a le droit de faire » du contrat du module.

Le calendrier du projet

Le projet est encadré par Fabien ESCOURBIAC.

Date	Horaire	Quoi	Où
mar 6 octobre	13 h 30 à 15 h 15	Lancement et premier suivi	à distance
jeu 15 octobre	journée entière	Cours : tests unitaires et pratiques professionnelles ; s'il reste du temps, travail sur le projet	en présentiel
mar 20 octobre	13 h 30 à 15 h 15	Second suivi	à distance
mer 4 novembre	13 h 30 à 17 h	**Soutenance**, devant Fabien ESCOURBIAC et l'enseignant du module	en présentiel, salles 315 et 316

Les cours du module continuent entre ces dates, les 21 et 23 octobre et le 3 novembre. L'ordre de passage des équipes aux deux suivis est donné par Fabien ESCOURBIAC.

Un défi d'accessibilité et d'audiodescription

Le projet s'inscrit dans un défi d'accessibilité, et plus précisément d'**audiodescription**, c'est-à-dire la restitution par le son d'une information qui n'est accessible que par la vue.

L'audiodescription existe déjà au cinéma, au théâtre et dans les musées : entre les dialogues, une voix décrit ce qui se passe à l'écran ou sur scène pour les personnes aveugles ou malvoyantes. Sur le web, l'équivalent minimal est le texte alternatif d'une image, l'attribut `alt`. Il est absent presque partout, ou réduit à « image.jpg ». Une personne qui navigue avec un lecteur d'écran ne sait alors pas ce que contient l'image.

Le défi consiste à fournir cette description à la demande : l'utilisateur transmet à l'application une image qu'il ne peut pas voir, et l'application la lui décrit.

Votre travail sera jugé en conséquence. Une application qui produit de bonnes descriptions mais qu'on ne peut pas utiliser sans voir l'écran ne répond pas au sujet. L'accessibilité doit donc être prise en compte dès le début de la conception de la page, et non ajoutée la dernière semaine.

Le sujet

Il s'agit de réaliser une application web, dans l'esprit d'un chatbot : l'utilisateur y dépose une image, un agent d'IA l'analyse, et l'application lit à voix haute la description de son contenu.

L'application s'adresse d'abord à une personne qui ne voit pas l'image. Celle-ci doit pouvoir l'utiliser du début à la fin sans l'aide d'une personne voyante : choisir le fichier, lancer l'analyse, entendre la description et la réécouter.

L'application a une interface graphique, avec des boutons, un sélecteur de fichier et des zones de texte ; elle ne se pilote pas uniquement à la voix. C'est cette interface qui doit être utilisable au clavier et avec un lecteur d'écran.

Une bonne audiodescription ne se limite pas à une liste d'objets. Elle donne d'abord l'essentiel (ce que représente l'image), puis les détails utiles, et elle lit le texte présent dans l'image : un panneau, une légende, un menu. C'est la consigne que vous donnez au modèle qui doit permettre d'obtenir ce résultat.

Ce qui est essentiel dépend aussi de ce que l'utilisateur cherche. La page comporte donc un champ texte facultatif, où il peut poser une question sur l'image : « Dans quelle rue suis-je ? », « Quelle est la date de péremption ? ». La question est envoyée avec l'image et oriente la réponse de l'agent. Sans question, l'agent donne la description générale.

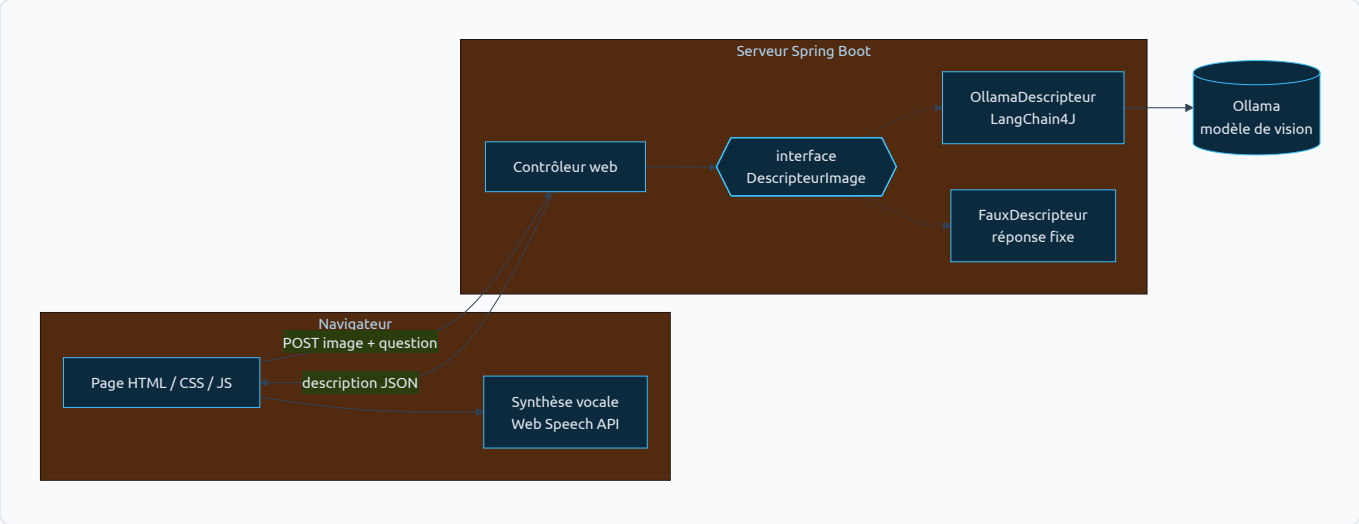
Le parcours minimal est le suivant :

1. l'utilisateur choisit une image sur son poste et, s'il le souhaite, écrit une question à son sujet ;
2. la page envoie l'image et la question au serveur ;
3. le serveur les confie à l'agent, qui renvoie en français une description de l'image ou la réponse à la question ;
4. la page affiche la réponse **et** la lit à voix haute ;
5. l'utilisateur peut relire, arrêter, recommencer avec une autre image.

Tout ce qui dépasse ce parcours (historique des images, conversation de suivi sur la même image, choix de la voix, plusieurs langues) est une extension. Commencez par faire fonctionner ces cinq étapes. Les extensions rapportent des points en plus, à condition que les cinq étapes fonctionnent et aient été validées : une extension ne compense pas une étape manquante.

Quand quelque chose échoue (fichier refusé, modèle arrêté, délai dépassé), la manière de le présenter est libre : un message lu à voix haute, un retour au début du parcours, une page dédiée. Elle doit convenir à un utilisateur qui n'est pas technicien et qui ne voit pas l'écran. Il doit comprendre ce qui s'est passé et ce qu'il peut faire ensuite, sans code HTTP ni trace d'exception, et le message doit lui parvenir par la voix ou par son lecteur d'écran.

L'architecture



L'application comporte trois parties, avec une seule règle de dépendance : le contrôleur ne connaît que l'interface. Il ignore l'existence d'Ollama et de LangChain4J.

Les outils

Côté	Outils	Rôle
`Serveur`	Java 17 ou plus, Gradle, Spring Boot (web)	recevoir l'image, la valider, appeler l'agent, renvoyer la réponse
`Agent`	LangChain4J	parler au modèle sans écrire soi-même les appels HTTP
`Modèle`	Ollama en local, ou un autre fournisseur	un modèle de vision, qui accepte une image en entrée : `llava`, `llama3.2-vision`, `gemma3`, `qwen2.5vl`…
`Page`	HTML, CSS, JavaScript, une bibliothèque si elle sert	envoyer le fichier, afficher la réponse
`Voix`	Web Speech API (`speechSynthesis`)	lire la description, dans le navigateur, sans serveur

> ⚠ ATTENTION
>
> Spring Boot 3 demande **Java 17** au minimum. Les ateliers du site utilisent Java 8 dans le navigateur, et votre projet ne peut pas s'y exécuter. Il se construit et se lance sur vos postes, avec `./gradlew bootRun`.

Contrainte technique : l'agent derrière une interface

C'est la seule contrainte imposée sur le code. Elle applique la séquence 3 à un projet réel (notion N10) : la partie agent est séparée du reste de l'application par une interface, de façon à pouvoir la remplacer sans modifier le contrôleur.

Changer d'agent correspond à l'un des cas suivants :

- passer d'Ollama à un autre fournisseur (un service en ligne, un autre serveur) ;
- passer d'un modèle à un autre, ou d'une consigne à une autre ;
- brancher une fausse implémentation qui répond une phrase fixe, pour développer la page, tester le contrôleur ou faire la démonstration quand le modèle est trop lent.

Si l'un de ces changements vous oblige à modifier le contrôleur, l'interface est mal conçue.

La fausse implémentation est obligatoire

Votre projet contient au moins deux implémentations de l'interface : celle qui appelle le modèle, et une fausse implémentation qui n'appelle aucun modèle et renvoie une réponse fixe ou prévisible. La fausse implémentation fait partie du rendu, au même titre que la vraie.

On doit pouvoir lancer l'application avec l'une ou l'autre sans modifier une ligne de Java : un profil Spring ou une propriété suffit. L'application doit alors fonctionner de bout en bout, page et synthèse vocale comprises, sur un poste où aucun modèle n'est installé. Le `README.md` indique comment la lancer ainsi.

Ce que le contrat contient

L'interface utilise les types de votre application et non ceux de la bibliothèque. Voici une esquisse ; ce n'est pas un corrigé, et vous choisissez les noms :

```java
public interface DescripteurImage {

    /**
     * Décrit le contenu d'une image en français, en quelques phrases,
     * ou répond à la question posée sur cette image.
     *
     * @param image les octets de l'image et son type MIME, jamais null
     * @param question la question de l'utilisateur, ou null pour une
description générale
     * @return la description ou la réponse, jamais vide
     * @throws DescriptionImpossibleException si l'agent ne peut pas répondre
     */
    Description decrire(Image image, String question) throws
DescriptionImpossibleException;
}
```

Vous devez prendre les décisions suivantes et savoir les justifier à l'oral :

1. **Les types utilisés dans la signature.** `Image` et `Description` sont des types que vous définissez. Si la signature mentionne `ImageContent`, `ChatResponse` ou `OllamaChatModel`, l'interface dépend de LangChain4J, et changer de bibliothèque obligera à modifier tout le code qui l'appelle.
2. **Les erreurs.** Un modèle arrêté, un délai dépassé ou une réponse vide sont des événements prévisibles, extérieurs à votre code. Faut-il une exception vérifiée ou non ? C'est la question traitée par la notion N14, ici dans un cas réel.
3. **Les préconditions.** Pour une image nulle, un fichier vide ou un type non pris en charge, qui fait la vérification, le contrôleur ou l'agent ? Une question vide équivaut-elle à une question absente ? Indiquez-le dans la Javadoc (notion N18).

> **⚠ PIÈGE CLASSIQUE**
>
> Une interface qui comporte une méthode par appel de LangChain4J ne fait que reproduire la bibliothèque. L'interface doit décrire ce dont le contrôleur a besoin (« décris-moi cette image ») et non la manière dont l'agent procède.

Injection de l'agent dans le contrôleur

L'interface ne suffit pas : un contrôleur qui écrit `new OllamaDescripteur()` dépend toujours d'Ollama. Le contrôleur doit recevoir l'agent par son constructeur, et c'est Spring qui le lui fournit :

```java
@RestController
public class DescriptionControleur {

    private final DescripteurImage descripteur;

    public DescriptionControleur(DescripteurImage descripteur) {
        this.descripteur = descripteur;
    }

    // ...
}
```

Le choix de l'implémentation se fait alors à un seul endroit : une classe `@Configuration`, un profil Spring (`@Profile("faux")`) ou une propriété dans `application.properties`. Pour changer d'agent, il suffit de modifier cette ligne.

▾ À QUOI RESSEMBLE L'APPEL À LANGCHAIN4J

Voici une esquisse pour LangChain4J 1.x avec Ollama. Ce code se trouve dans l'implémentation, jamais dans le contrôleur :

```java
ChatModel modele = OllamaChatModel.builder()
        .baseUrl("http://localhost:11434")
        .modelName("llava")
        .timeout(Duration.ofMinutes(2))
        .build();

UserMessage message = UserMessage.from(
        TextContent.from("Décris cette image en français, en trois
phrases, pour une personne qui ne la voit pas."),
        ImageContent.from(base64, typeMime));

String texte = modele.chat(message).aiMessage().text();
```

Vérifiez les noms dans la version que vous installez : l'API de LangChain4J a changé entre les versions 0.x et 1.x, et un assistant vous proposera souvent l'ancienne.

On reconnaît ici le patron Adaptateur (chapitre 10) : `OllamaDescripteur` adapte une bibliothèque à l'interface attendue par votre code. Vous pouvez utiliser d'autres patrons, à condition d'expliquer à l'oral quel problème chacun résout (notion N12).

Le binôme

Le projet se fait à deux. Les équipes sont fixées le 6 octobre.

Chacun écrit du Java. On ne peut pas confier le Java à l'un et la page à l'autre. Voici un exemple de répartition :

A	**l'agent : l'implémentation LangChain4J, la consigne envoyée au modèle, la fausse implémentation**
B	le web : le contrôleur, la validation du fichier reçu, la traduction des erreurs en réponses HTTP

L'interface sert de point de rencontre entre les deux parties : écrivez-la ensemble, en premier. Chacun peut ensuite avancer de son côté ; B travaille avec la fausse implémentation, sans attendre que le modèle fonctionne.

À la soutenance, chacun présente sa partie Java sans l'assistant (notion N27), et peut être interrogé sur celle de l'autre. L'historique Git montre qui a écrit quoi : faites vos commits à votre nom, plutôt que de tout pousser depuis un seul poste.

Le rendu et la soutenance

Ce qui est rendu

- Le code, dans un dépôt GitHub, avec l'historique des commits de chacun.
- Les deux implémentations de l'agent, la vraie et la fausse.
- Au moins un `README.md` complet : ce que fait l'application, comment l'installer et la lancer, quel modèle elle attend, et comment passer à la fausse implémentation.

Le rendu se fait sur GitHub, le mercredi 4 novembre. La soutenance a lieu le même jour, de 13 h 30 à 17 h, en salles 315 et 316. L'ordre de passage et la durée par équipe seront précisés sur cette page.

Le déroulement de la soutenance

1. Une courte présentation, avec quelques diapositives, sur la façon dont vous avez conduit le projet. Elle reste courte, pour laisser le temps à la démonstration.
2. La démonstration de l'application, qui est la partie la plus importante.

3. Les questions du jury, auxquelles chacun répond sans l'assistant.

Ce qui est regardé

- L'application répond-elle aux questions portant sur l'image déposée ?
- L'interface est-elle accessible ?
- Les patrons sont-ils appliqués là où ils servent ?
- Le code suit-il les bonnes pratiques de développement ?
- L'usage de Git : commits réguliers, au nom de leur auteur.
- La répartition du travail dans le binôme.

Difficultés fréquentes

> ⚠ ATTENTION
>
> **Le modèle est lent et volumineux.** Un modèle de vision de 7 milliards de paramètres pèse environ 5 Go et demande 8 Go de mémoire libre ; sans carte graphique, une description prend plusieurs dizaines de secondes. Prévoyez un délai d'attente réglable, un message d'attente sur la page, et utilisez la fausse implémentation pour le reste du développement.

- **Spring refuse les fichiers de plus de 1 Mo par défaut.** Une photo prise avec un téléphone dépasse cette taille. Réglez `spring.servlet.multipart.max-file-size` et `max-request-size`, et fixez une limite plutôt que de la supprimer.
- **Le nom du fichier n'indique pas son type.** Un fichier `.jpg` peut contenir n'importe quoi. Vérifiez le type annoncé et refusez les formats que le modèle ne sait pas lire.
- **La lecture ne démarre pas automatiquement.** Plusieurs navigateurs bloquent la synthèse vocale tant que l'utilisateur n'a pas cliqué ou appuyé sur une touche dans la page. Déclenchez la lecture à partir d'une action de l'utilisateur, et prévoyez un bouton « relire ».
- **Les voix disponibles varient selon le navigateur.** Choisissez une voix `fr-FR` si elle est disponible, sans supposer qu'elle l'est toujours.

- **La page doit être utilisable sans souris.** Tout doit être accessible au clavier, les contrôles doivent avoir des étiquettes, et la description écrite doit se trouver dans une zone `aria-live` pour qu'un lecteur d'écran l'annonce aussi.
- **Le modèle peut inventer.** Il lui arrive de décrire un objet absent de l'image. La formulation de votre consigne peut rendre ces erreurs plus ou moins fréquentes ; testez-la sur des images dont vous connaissez le contenu.

Notions mobilisées par le projet

Notion	Où elle intervient dans le projet
N10 Dépendre d'une interface	le contrat de l'agent, et le contrôleur qui le reçoit
N9 Couplage et cohésion	aucune classe de LangChain4J hors de l'implémentation de l'agent
N12 Choisir un patron, ou n'en pas mettre	l'adaptateur autour de LangChain4J, et les autres que vous ajouterez
N14, **N17** Exceptions	l'agent qui ne répond pas, et ce que la page en affiche
N18 Préconditions	le fichier reçu : taille, type, présence
N26 Relire le code de l'assistant	tout ce qu'il vous proposera sur LangChain4J, qui change vite
N27 Justifier à l'oral	chaque décision ci-dessus, sans l'assistant